CHEATSHEET

SQL

ESTRUCTURA BÁSICA Y FILTRADO

- **SELECT**: Selecciona las columnas que queremos obtener de la tabla. Para traer todas las columnas se usa *
- **FROM**: Indica de qué tabla principal queremos tomar los datos
- **WHERE**: Filtra los datos basándose en condiciones antes de agrupar
- **AND**: Operador lógico donde todas las condiciones deben cumplirse al mismo tiempo
- **OR**: Operador lógico donde basta con que al menos una condición se cumpla
- **DISTINCT**: Elimina las filas duplicadas del resultado
- **AS**: Crea un alias para renombrar una columna o tabla
- **BETWEEN**: Verifica si un valor está dentro de un rango (incluyendo extremos)
- **LIMIT**: Limita la cantidad de filas que devuelve la consulta
- **OFFSET**: Salta una cantidad específica de filas antes de empezar a mostrar resultados

ORDENAMIENTO Y AGRUPACIÓN

- **ORDER BY**: Ordena los resultados. Por defecto es ascendente (ASC)
- **DESC**: Ordena de forma descendente (de mayor a menor)
- **GROUP BY**: Agrupa filas que tienen el mismo valor en columnas específicas para aplicarles funciones de agregación
- **HAVING**: Funciona igual que WHERE pero filtra *después* de que se agrupan los datos, permitiendo usar funciones de agregación

MANEJO DE NULOS

- **NULL**: Representa la ausencia de valor. No es un cero ni texto vacío
- **IS NULL**: Expresión especial para verificar si una celda está vacía
- **IS NOT NULL**: Verifica que una celda sí contenga algún dato

- **IFNULL(val1, val2)**: Si el primer valor es NULL, devuelve el segundo valor
- **COALESCE(val1, val2, ...)**: Recibe una lista y devuelve el primer valor que no sea NULL

UNIONES (JOINs)

- **LEFT JOIN**: Toma todos los registros de la tabla izquierda y los une con las coincidencias de la derecha. Si no hay coincidencia, rellena con NULL
- **INNER JOIN** (o JOIN): Solo devuelve los registros que tienen coincidencia en ambas tablas
- **RIGHT JOIN**: Mantiene todos los registros de la tabla de la derecha, rellenando con NULL la izquierda si no hay coincidencia
- **CROSS JOIN**: Genera todas las combinaciones posibles entre las filas de dos tablas
- **Self JOIN**: Unir una tabla consigo misma usando alias distintos para comparar filas internamente
- **ON**: Define por qué columnas se van a conectar las tablas
- **USING(col)**: Atajo para ON cuando la columna se llama exactamente igual en ambas tablas

FUNCIONES DE AGREGACIÓN

- **COUNT(*)**: Cuenta todas las filas de un grupo
- **COUNT(columna)**: Cuenta las filas de una columna ignorando los valores NULL
- **COUNT(DISTINCT columna)**: Cuenta solamente los valores únicos de esa columna
- **SUM()**: Suma los valores numéricos de un grupo. También puede sumar condiciones booleanas (ej. SUM(rating < 3))
- **AVG()**: Calcula el promedio simple. Para promedio ponderado usar SUM(val1*val2)/SUM(val2)
- **MIN()** / **MAX()**: Devuelven el valor mínimo y máximo de una columna
- **GROUP_CONCAT()**: Une varios valores de texto en una sola cadena. Permite usar DISTINCT, ORDER BY y SEPARATOR en su interior

FUNCIONES DE TEXTO Y BÚSQUEDA

- **LENGTH()**: Devuelve la cantidad de caracteres de un texto
- **SUBSTRING(texto, inicio, long)**: Extrae una parte de un texto
- **UPPER()**: Convierte texto a mayúsculas
- **LOWER()**: Convierte texto a minúsculas
- **CONCAT(t1, t2)**: Une o pega varios fragmentos de texto en uno solo.
- **LIKE**: Busca coincidencias parciales usando el comodín % (representa cualquier cantidad de caracteres)

- **REGEXP**: Permite usar expresiones regulares para validar patrones complejos (ej. ^[a-zA-Z] para inicios, $ para finales)
- **BINARY**: Fuerza a que comparaciones como LIKE distingan estrictamente entre mayúsculas y minúsculas

FUNCIONES MATEMÁTICAS Y FECHAS

- **ROUND(valor, decimales)**: Redondea un número a la cantidad de decimales indicada
- **Módulo (%)**: x % y obtiene el residuo de una división. Útil para pares (x % 2 = 0) o impares (x % 2 = 1)
- **DATE_ADD(fecha, INTERVAL n DAY)**: Suma un intervalo de tiempo a una fecha.
- **DATE_FORMAT(fecha, '%Y-%m')**: Cambia el formato de una fecha (ej. de YYYY-MM-DD a YYYY-MM)

CONTROL DE FLUJO

- **CASE WHEN ... THEN ... ELSE ... END**: Evalúa una condición como un if/else. Retorna THEN si es verdadera y ELSE si es falsa

SUBCONSULTAS Y TABLAS TEMPORALES

- **Subconsulta en WHERE**: Para filtrar filas comparándolas contra otra consulta (con IN o NOT IN)
- **Subconsulta en FROM**: Crea una tabla derivada temporal; obligatoriamente lleva alias
- **Subconsulta en SELECT**: Retorna un solo valor calculado. Si no encuentra nada, devuelve NULL
- **Subconsulta Correlacionada**: Una subconsulta que usa datos de la fila actual de la consulta externa, ejecutándose una vez por cada fila
- **CTE (WITH ... AS)**: Crea tablas temporales al inicio del código para mayor legibilidad
- **UNION**: Apila resultados de varias consultas eliminando duplicados
- **UNION ALL**: Apila resultados de varias consultas manteniendo todos los duplicados

WINDOW FUNCTIONS (FUNCIONES DE VENTANA)

Realizan cálculos usando filas relacionadas sin colapsarlas (como hace GROUP BY)

- **OVER(ORDER BY col)**: Define la "ventana" o el orden interno para la función
- **PARTITION BY**: Divide las filas en grupos dentro de la ventana para reiniciar el cálculo en cada grupo
- **LAG(columna, n)**: Accede al valor de una fila anterior
- **ROW_NUMBER()**: Asigna un número secuencial (1, 2, 3...) a las filas de la partición
- **Suma acumulativa**: SUM(col) OVER (ORDER BY col) va sumando el valor actual más los anteriores
- **Ventana móvil**: ROWS BETWEEN n PRECEDING AND CURRENT ROW restringe el cálculo a las últimas *n* filas más la actual

MODIFICACIÓN DE DATOS

- **DELETE**: Elimina filas permanentemente. Si se usa con JOIN, se debe especificar qué alias de tabla se va a borrar (ej. DELETE p1 FROM tabla p1 JOIN...)

REPOSITORIO GITHUB

CANAL DE YOUTUBE